

Handwritten Digit Classification Using Convolutional Neural Networks (CNN) on the MNIST Dataset

Submitted By: PRANEETH REDDY
College: Atria University
Organization: Navitech Solution Pvt. Ltd.
Submission Date: August 2026

Acknowledgement

I would like to express my sincere gratitude to **Navitech Solution Pvt. Ltd.** for providing me with the opportunity to complete this internship project. This project has helped me gain practical knowledge of Deep Learning, Image Classification, Python programming, TensorFlow, and Keras.

I also thank my mentor and my college faculty for their continuous guidance and encouragement throughout the internship. Their valuable suggestions and support played an important role in the successful completion of this project.

Abstract

Handwritten digit recognition is one of the most popular applications of deep learning and computer vision. In this project, a Convolutional Neural Network (CNN) is developed to classify handwritten digits using the MNIST dataset. The dataset contains thousands of grayscale images of digits from 0 to 9.

The project includes loading the dataset, preprocessing the images by normalization, building a CNN model, training the model, evaluating its performance, and predicting handwritten digits. The trained model achieves high accuracy, demonstrating the effectiveness of CNNs in image classification tasks.

Table of Contents

- | | |
|--------------------------|------------------------|
| 1. Introduction | 4. Dataset Description |
| 2. Objectives | 5. Methodology |
| 3. Software Requirements | 6. CNN Architecture |

7. Model Training

8. Results

9. Advantages

10. Applications

11. Conclusion

12. Future Scope

13. References

1. Introduction

Handwritten digit recognition is an important problem in machine learning and computer vision. It is used in applications such as bank cheque processing, postal code recognition, document digitization, and optical character recognition (OCR).

Convolutional Neural Networks (CNNs) are powerful deep learning models that automatically learn image features without manual feature extraction. In this project, TensorFlow and Keras are used to build a CNN model capable of recognizing handwritten digits with high accuracy.

2. Objectives

The objectives of this project are:

- To understand the MNIST dataset.
- To preprocess image data.
- To build a CNN model.
- To train the model using TensorFlow and Keras.
- To evaluate model performance.
- To predict handwritten digits accurately.

3. Software Requirements

Hardware

- Computer / Laptop
- Internet Connection

Software & Libraries

- Python
- Google Colab
- TensorFlow & Keras
- NumPy, Matplotlib, Scikit-learn

4. Dataset Description

The MNIST dataset contains handwritten digit images collected from thousands of writers. Each image is 28×28 pixels in grayscale format.

- **Training Samples:** 60,000 images

- **Testing Samples:** 10,000 images
- **Dimensions:** 28×28 pixels
- **Output Classes:** 10 classes (digits 0–9)

The images are normalized before training so that the neural network can learn more efficiently.

5. Methodology

The project follows these steps:

1. Import required libraries.
2. Load the MNIST dataset.
3. Normalize image pixel values.
4. Convert labels into categorical format (if used).
5. Build the CNN architecture.
6. Compile the model.
7. Train the model.
8. Evaluate the trained model.
9. Predict handwritten digits.
10. Save the trained model.

6. CNN Architecture

The Convolutional Neural Network consists of:

- **Input Layer:** Accepts $28 \times 28 \times 1$ image tensors.
- **Convolution Layer:** Extracts image features using spatial filters.
- **ReLU Activation:** Introduces non-linearity into the network.
- **Max Pooling Layer:** Reduces feature map dimensionality.
- **Flatten Layer:** Reshapes spatial feature maps into a 1D vector.
- **Dense Hidden Layer:** Fully connected layer for representation learning.
- **Softmax Output Layer:** Outputs probability distribution across 10 digit classes.

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204928
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1290

=====
Total params: 225,034 (879.04 KB)
Trainable params: 225,034 (879.04 KB)
Non-trainable params: 0 (0.00 B)

7. Model Training

The CNN model is trained using the training dataset. The training parameters used are:

- **Optimizer:** Adam
- **Loss Function:** Categorical Cross-Entropy
- **Batch Size:** 128
- **Epochs:** 10

The training process demonstrates continuous loss reduction and accuracy improvement across epochs:

```
Phase 4: Training model...
Epoch 1/10 - accuracy: 0.9218 - loss: 0.2580 - val_accuracy: 0.9828 - val_loss: 0.0603
Epoch 2/10 - accuracy: 0.9773 - loss: 0.0758 - val_accuracy: 0.9860 - val_loss: 0.0478
Epoch 3/10 - accuracy: 0.9825 - loss: 0.0562 - val_accuracy: 0.9887 - val_loss: 0.0387
Epoch 4/10 - accuracy: 0.9868 - loss: 0.0429 - val_accuracy: 0.9888 - val_loss: 0.0369
Epoch 5/10 - accuracy: 0.9892 - loss: 0.0347 - val_accuracy: 0.9915 - val_loss: 0.0313
Epoch 6/10 - accuracy: 0.9903 - loss: 0.0304 - val_accuracy: 0.9902 - val_loss: 0.0339
Epoch 7/10 - accuracy: 0.9913 - loss: 0.0266 - val_accuracy: 0.9938 - val_loss: 0.0280
Epoch 8/10 - accuracy: 0.9930 - loss: 0.0222 - val_accuracy: 0.9895 - val_loss: 0.0337
Epoch 9/10 - accuracy: 0.9937 - loss: 0.0192 - val_accuracy: 0.9892 - val_loss: 0.0390
Epoch 10/10 - accuracy: 0.9948 - loss: 0.0187 - val_accuracy: 0.9923 - val_loss: 0.0327
```

8. Results

After training, the model is evaluated using the test set (10,000 images):

Phase 5: Evaluating model performance

- **Test Accuracy:** 99.21%
- **Test Loss:** 0.0300

Classification Performance Summary

Class (Digit)	Precision	Recall	F1-Score	Support
0	0.99	1.00	0.99	980
1	0.99	1.00	1.00	1135
2	1.00	0.99	0.99	1032
3	1.00	0.99	0.99	1010
4	0.99	0.99	0.99	982
5	0.99	0.98	0.99	892
6	0.99	0.99	0.99	958
7	1.00	0.99	0.99	1028
8	0.99	0.99	0.99	974
9	0.99	0.98	0.99	1009
Accuracy / Overall	0.99	0.99	0.99	10000

9. Advantages

- High classification accuracy (>99%).
- Automatic feature extraction without manual engineering.
- Fast inference/prediction speed after training.
- Highly suitable for Optical Character Recognition (OCR) systems.
- Easy implementation and scalability using Keras and TensorFlow.

10. Applications

- Bank cheque processing and automated amount verification.
- Postal code and ZIP code recognition on mail envelopes.
- Optical Character Recognition (OCR) in document digitization.
- Digitization of historical handwritten records and documents.
- Educational and academic deep learning benchmarks.
- AI-based vision recognition in mobile and embedded systems.

11. Conclusion

This project successfully implemented a Convolutional Neural Network for handwritten digit recognition using the MNIST dataset. The trained model achieved high classification accuracy (99.21%) and accurately predicted handwritten digits across all test classes. The project provided practical hands-on experience with TensorFlow, Keras, and Google Colab while demonstrating the core effectiveness of deep learning for computer vision tasks.

12. Future Scope

The current project can be extended and enhanced in several ways:

- Increasing dataset size and variety through advanced Data Augmentation (rotation, scaling, noise).
- Testing deeper architectural models like ResNet or EfficientNet for enhanced robustness.
- Deploying the model as a live web application using Flask/FastAPI.
- Developing a real-time mobile application for camera-based digit recognition.

13. References

1. TensorFlow Documentation — <https://www.tensorflow.org>
2. Keras API Reference — <https://keras.io>
3. The MNIST Database of Handwritten Digits (Yann LeCun)
4. NumPy Documentation — <https://numpy.org>
5. Matplotlib Graphical Library — <https://matplotlib.org>
6. Scikit-learn Machine Learning Library — <https://scikit-learn.org>